

ABC Company

Technology Stack — Deep Dive

Document ID:	ABC-DOC-004
Version:	v1.0
Date:	June 2026
Prepared By:	ABC Company — Engineering
Classification:	Business Confidential

ABC Company — Colombo | London | Melbourne

Table of Contents

1	Technology Philosophy	
2	Frontend Frameworks	
3	Mobile Development	
4	Backend Frameworks	
5	Databases	
6	Cloud & Infrastructure	
7	AI/ML Stack	
8	Testing & Quality	
9	Monitoring & Observability	

1. Technology Philosophy

ABC Company's approach to technology selection is grounded in a single, defining principle: the right tool for the right job. There is no single framework, language, or platform that suits every project, every client, or every engineering team. Technology decisions at ABC Company are driven by a structured evaluation of project requirements, team expertise, the client's existing ecosystem, and the long-term maintainability of the resulting system. Choosing the fashionable option over the appropriate option is considered an engineering failure, not a feature.

ABC Company operates an open-source-first policy. The default assumption is that an open-source solution exists and should be evaluated before a commercial alternative is considered. Commercial tools are adopted where they provide clear, demonstrable value that cannot be reasonably replicated in-house: managed availability, enterprise support SLAs, compliance certifications, or tightly integrated vendor ecosystems where integration cost would exceed commercial licensing cost. Every commercial tool in the stack must be reviewed annually to ensure its continued justification.

Quarterly technology radar reviews are conducted by the Engineering Leadership team, drawing participation from principal engineers and tech leads across all practice areas. The radar categorises technologies into four rings: Adopt (production-ready, recommended), Trial (suitable for low-risk production use), Assess (under evaluation, prototype use only), and Hold (deprecated, not to be started on new projects). The radar output is communicated to all engineering staff and directly informs staffing, training investment, and partner decisions.

Adoption conservatism is a deliberate policy. ABC Company waits for at least a stable 1.0 release, observable community maturity (measured by package download trends, GitHub activity, and availability of qualified engineers in the hiring market), and ecosystem stability before committing to a new framework on client work. Early-adopter risk is reserved for internal tooling and R&D projects where the blast radius is contained. This conservatism protects clients from being stranded on unmaintained dependencies and protects the firm from skills cliffs when key team members rotate off a project.

Standardisation and flexibility are held in balance through a tiered recommendation system. Every practice area (frontend, mobile, backend, data) maintains a default stack — the combination of tools that any qualified engineer joining the project should be productive with in their first sprint. Deviations from the default stack are permitted, but they require documented justification and sign-off from the relevant practice lead. Justification must address: why the default stack fails to meet the requirement, what the onboarding cost of the alternative is, and what the exit strategy looks like if the alternative technology is later deprecated.

2. Frontend Frameworks

2.1 React (Primary Framework)

React is ABC Company's default choice for web application development and is used in approximately 80% of client-facing web projects. The team comprises fifteen frontend engineers with deep, production-hardened React expertise spanning from 2018 to the present day, covering the full evolution from class components through hooks-based functional architecture. React's component-driven model aligns naturally with ABC Company's design system, enabling consistent reuse of UI primitives across projects and clients. State management strategy is tiered: React Query (TanStack Query) handles all server state, asynchronous fetching, caching, and background revalidation; Zustand is used for lightweight client-side global state; Redux Toolkit is reserved for applications with complex, interdependent global state trees that benefit from the predictability of a single source of truth and time-travel debugging. Performance optimisation is treated as a first-class concern, with code splitting via React.lazy and Suspense, route-level lazy loading, useMemo and useCallback applied judiciously, and bundle analysis integrated into the CI pipeline. Testing follows the React Testing Library philosophy of testing behaviour over implementation, paired with Jest for the test runner and assertion layer.

2.2 Next.js

Next.js is the framework layer used on all new React projects that require search engine optimisation, server-side rendering, or static site generation. The team transitioned to the App Router (introduced in Next.js 13) as the default architecture, leveraging React Server Components to push data-fetching logic to the server and dramatically reduce the client-side JavaScript bundle. Incremental Static Regeneration (ISR) is used extensively for content-heavy pages where fresh data is required but full SSR on every request would be prohibitively expensive. API Routes and Route Handlers serve as a thin backend-for-frontend layer, handling authentication callbacks, form submissions, and webhook ingestion. Next.js Middleware is used for edge-level authentication guards, locale redirection, and A/B testing header injection. Deployment targets are Vercel for client-managed infrastructure and self-hosted Dockerised deployments on AWS ECS or Kubernetes for enterprise clients with strict data-residency requirements.

2.3 Angular

Angular is adopted when the client's internal development team already operates on the Angular ecosystem, or when a large-scale enterprise application demands the opinionated, convention-heavy architecture that Angular provides. Angular's dependency injection system, strong TypeScript typing, and CLI-driven scaffolding reduce architectural decision fatigue on large teams where consistency across dozens of contributors is more important than developer flexibility. RxJS is used throughout for asynchronous data flows, though ABC Company's Angular projects enforce a strict policy of limiting reactive streams to service boundaries — components consume observables but do not create them, reducing the risk of subscription leaks. NgModules are being migrated to the standalone component model on all new Angular 17+ projects. ABC Company maintains four Angular-specialist frontend engineers.

2.4 Vue.js

Vue.js is used for smaller scoped projects, rapid prototypes, and scenarios where a client's existing team or technology preference makes Vue the natural fit. The Composition API is standard on all Vue 3 projects, providing a patterns-alignment with React hooks that simplifies context switching for engineers who work across both ecosystems. Pinia replaces Vuex as the state management library on all new Vue projects. Vue's progressive

adoption model makes it a pragmatic choice for adding interactivity to server-rendered pages without a full single-page application architecture commitment.

2.5 Svelte

Svelte occupies the Assess ring of ABC Company's technology radar. It is used experimentally on internal tooling and performance-critical lightweight interfaces — dashboards and widgets where the absence of a runtime framework overhead produces measurably better performance. Two engineers have completed Svelte projects in a production context; the framework has not yet been adopted for client-facing work at scale. Its promotion to the Trial ring is under evaluation following the SvelteKit 1.0 stable release.

2.6 Framework Comparison

Framework	Primary Use Case	Learning Curve	Bundle Size	Ecosystem Maturity	ABC Team Expertise
React	Primary web app framework	Medium	Moderate (tree-shakable)	Excellent	Expert (15 engineers)
Next.js	SSR / SSG / full-stack React	Medium–High	Moderate	Excellent	Expert
Angular	Large enterprise apps, client-ecosystem match	High	Large	Excellent	Proficient (4 engineers)
Vue.js	Small–medium apps, prototypes	Low–Medium	Small	Good	Proficient (3 engineers)
Svelte	Lightweight perf-critical UIs	Low	Very Small	Growing	Emerging (2 engineers)

3. Mobile Development

3.1 React Native with Expo (Primary Cross-Platform)

React Native with Expo is ABC Company's default mobile development platform and is used on approximately 70% of mobile projects. The primary driver is code-sharing efficiency: teams already proficient in React can contribute to mobile codebases with minimal ramp-up time, and shared business logic, validation, and API client code can be extracted into platform-agnostic packages consumed by both web and mobile. Expo's managed workflow reduces the operational overhead of native build toolchain management, while EAS Build provides cloud-based iOS and Android builds integrated directly into CI/CD pipelines. Over-the-Air (OTA) updates via Expo Updates allow JavaScript-layer bug fixes and feature releases to reach users without requiring App Store or Google Play review cycles, a significant commercial advantage for fast-moving product teams. When managed workflow limitations are encountered — typically for deep hardware integrations, custom native camera behaviour, or Bluetooth peripherals — ABC Company employs a Continuous Native Generation strategy using Expo's bare workflow, allowing custom native modules to be bridged while retaining the Expo toolchain for the JavaScript layer. Eight mobile engineers carry React Native expertise on the team.

3.2 Flutter

Flutter is chosen when the project's design requirements demand native-quality animations, highly custom UI components that are difficult to achieve with React Native's bridge model, or when a single Dart codebase targeting mobile, web, and desktop is commercially justified. Flutter's widget-based rendering architecture draws every pixel independently of the underlying platform, guaranteeing pixel-perfect consistency across iOS and Android — a property that matters greatly on brand-identity-driven consumer applications. ABC Company's Flutter engineers work with both Material and Cupertino design systems and apply Riverpod as the primary state management solution. The Dart language, while niche, is well-regarded for its strong typing, null safety, and approachability. ABC Company maintains three Flutter specialists.

3.3 Swift (iOS Native)

Swift with SwiftUI is used exclusively for iOS-only projects where deep Apple platform integration is a hard requirement. Typical use cases include HealthKit integration for fitness and medical applications, ARKit and RealityKit for augmented reality experiences, Core ML for on-device machine learning inference, and applications with complex, platform-native interaction models (e.g., Live Activities, Dynamic Island, Widget extensions) that cross-platform frameworks cannot replicate cleanly. UIKit remains used for legacy codebases and complex custom animations where SwiftUI's layout engine constraints create friction. Combine is used for reactive data flows on UIKit projects; SwiftUI's native bindings (`@State`, `@ObservableObject`, `@Observable` in Swift 5.9) are preferred on SwiftUI projects.

3.4 Kotlin (Android Native)

Kotlin with Jetpack Compose is ABC Company's choice for Android-only projects that mirror the iOS use case: applications requiring access to Android-specific APIs (Google Pay deep integration, Android Auto, Wear OS, or NFC peripheral control) that cross-platform abstractions do not surface adequately. Jetpack Compose provides a modern, declarative UI paradigm analogous to SwiftUI, enabling teams with cross-platform backgrounds to transition to native Android work with manageable cognitive load. Kotlin Coroutines and Flow form the concurrency and reactive data layer, and Room is used as the local SQLite abstraction for offline-first architectures.

3.5 Platform Decision Matrix

Criterion	React Native + Expo	Flutter	Native (Swift / Kotlin)
Project targets both iOS & Android	Yes	Yes (if custom UI critical)	No
Shares logic with a React web app	Yes (strong fit)	No	No
Custom UI animations are paramount	Possible	Yes (best choice)	Possible
Deep Apple platform APIs (HealthKit, ARKit)	No	No	Yes
Deep Android platform APIs (NFC, Wear OS)	No	No	Yes (Android)
Team is React-proficient	Yes (lowest ramp-up)	No	No
Pixel-perfect cross-platform consistency	Partial	Yes	N/A
OTA updates required	Yes	Limited	No

4. Backend Frameworks

4.1 Node.js / NestJS (Primary)

Node.js with NestJS is ABC Company's default backend framework and underpins approximately 60% of server-side projects. The choice of NestJS over a minimal framework such as Express or Fastify is intentional and architectural: NestJS enforces a modular, dependency-injection-driven application structure that scales gracefully from a single-service monolith to a distributed microservices system without requiring a large-scale refactor at transition points. TypeScript is the default language across all NestJS projects, providing end-to-end type safety when paired with Prisma ORM for database access and tRPC or GraphQL for API contracts. NestJS's decorator-based controller and service pattern is familiar to engineers with Spring Boot or .NET backgrounds, reducing onboarding friction on mixed-background teams. Microservices communication is handled via NestJS's built-in transporter layer supporting RabbitMQ, Kafka, and Redis pub/sub. All NestJS projects include unit tests for service classes (Jest), e2e tests for HTTP endpoints (Supertest), and integration tests for database interactions using a Dockerised PostgreSQL instance in CI.

4.2 Python / FastAPI / Django

Python forms the backbone of ABC Company's AI, data engineering, and analytics practice. Of the eighteen backend engineers on the team, approximately forty percent carry strong Python proficiency. FastAPI is the preferred framework for all new Python API services: its native async support, automatic OpenAPI documentation generation, and Pydantic-based request and response validation dramatically reduce the boilerplate cost of building well-documented, type-safe APIs. FastAPI's dependency injection system, while less opinionated than NestJS's IoC container, is sufficient for the services it is applied to. Django is reserved for projects that benefit from its included-batteries philosophy: multi-user admin interfaces, complex authentication and permission systems, ORM-driven data management tools, and projects where the client's in-house team maintains the codebase long-term and values Django's stability and extensive documentation. SQLAlchemy with Alembic is used for database management on FastAPI projects; Django ORM is used natively on Django projects.

4.3 Go

Go is used for performance-critical microservices, command-line tooling, and infrastructure-adjacent components where low memory footprint and predictable latency profiles are requirements. Goroutines and channels provide a concurrency model that maps cleanly to high-throughput request-handling and stream-processing scenarios. Go's compilation to a single, statically linked binary with no runtime dependency simplifies containerisation — Go services produce the smallest Docker images in ABC Company's portfolio, typically under 20 MB using multi-stage builds. Go is used in ABC Company's internal observability tooling, log aggregation agents, and two client-facing high-performance data ingestion APIs. Four backend engineers hold Go proficiency.

4.4 .NET / ASP.NET Core

.NET with C# and ASP.NET Core is adopted when client infrastructure is deeply embedded in the Microsoft ecosystem — Azure-native deployments, Active Directory identity integration, SQL Server as the primary database, or existing in-house .NET teams whose knowledge transfer from an external partner is commercially critical. ABC Company maintains two .NET specialists and access to a network of .NET contractors for larger engagements. Entity Framework Core is the ORM of choice; SignalR is used for real-time communication features; Minimal APIs are preferred over the MVC controller model for new services.

4.5 Backend Framework Comparison

Framework	Primary Use Case	TypeScript Support	Key Characteristics	ABC Team Expertise
NestJS (Node.js)	TypeScript-first APIs, microservices, GraphQL	Medium	Strong (DI, decorators, modules)	Expert (18 engineers)
FastAPI (Python)	AI/ML APIs, data services, rapid prototyping	Low	Async-native, auto-docs, Pydantic	Proficient (7 engineers)
Django (Python)	Admin-heavy, ORM-driven web apps, full-featured	Medium	Batteries included, ORM, auth	Proficient (5 engineers)
Go	High-perf microservices, CLI tools, infra components	Medium–High	Minimal, single binary, goroutines	Proficient (4 engineers)
.NET / ASP.NET Core	Microsoft-ecosystem client work	Medium	Enterprise-grade, mature	Specialist (2 engineers)

5. Databases

5.1 PostgreSQL (Primary)

PostgreSQL is ABC Company's default relational database and is used on approximately 85% of projects. Its choice is rarely a decision — PostgreSQL is simply the starting point, and deviations require justification. The reasons for this default are well established: full ACID compliance, a mature MVCC concurrency model, JSON and JSONB column types for semi-structured data without sacrificing relational query capability, PostGIS for geospatial use cases, full-text search via tsvector and tsquery, window functions and CTEs for complex analytical queries that would otherwise require application-level processing, and a rich ecosystem of migration tools. Alembic is used for schema migrations on FastAPI/SQLAlchemy projects; Prisma Migrate is used on NestJS projects. All PostgreSQL deployments use RDS Multi-AZ on AWS for production, with pgBouncer for connection pooling. Read replicas are provisioned for reporting and analytics workloads to avoid contention with transactional traffic.

5.2 MongoDB

MongoDB is used for document-oriented data storage where a flexible, schema-light model is genuinely appropriate — content management systems, event logging, product catalogues with heterogeneous attribute sets, and applications whose data shape is expected to evolve rapidly in early product development phases. The key discipline applied in MongoDB projects is intentional schema design: ABC Company does not treat MongoDB as schema-free in practice. Schemas are enforced at the application layer using Mongoose validation or Pydantic model serialisation, and JSON Schema validation rules are applied at the database collection level. The anti-pattern of using MongoDB simply because a team wants to avoid writing SQL is actively discouraged.

5.3 Redis

Redis is a component in nearly every ABC Company production architecture, though its role varies. For caching, Redis serves as the L2 cache layer in front of PostgreSQL for expensive query results, with TTL strategies calibrated to data volatility. For session storage, Redis provides the distributed session backend for horizontally scaled API services. Rate limiting is implemented via Redis sorted sets and the Lua-scripted sliding window algorithm, providing atomic, distributed rate enforcement without external coordination. Pub/sub and Redis Streams are used for lightweight intra-service event propagation where a full Kafka deployment would be operationally disproportionate. Redis Cluster is provisioned for workloads requiring horizontal write scalability or datasets exceeding the memory capacity of a single node. ElastiCache for Redis is the standard AWS deployment target.

5.4 Elasticsearch

Elasticsearch is used when full-text search quality, faceted filtering, or log aggregation at scale drives the requirement. Elasticsearch's inverted index architecture delivers relevance-ranked search results with sub-second latency across billions of documents — a capability that PostgreSQL's full-text search approximates at small scale but cannot match at large scale. Aggregation pipelines in Elasticsearch serve as a real-time analytics layer for dashboards that query log and event data. Kibana provides the default visualisation and exploration interface. ABC Company uses the Elastic Cloud managed service for client deployments to avoid the operational burden of cluster management, shard allocation, and index lifecycle policy tuning.

5.5 DynamoDB

DynamoDB is used exclusively within AWS-native, serverless architectures where the application's access patterns are well-defined and throughput requirements are high or bursty. DynamoDB's single-table design pattern is adopted by default — ABC Company's DynamoDB projects define a careful partition key and sort key schema upfront, with Global Secondary Indexes designed to support all required access patterns. The distinction between on-demand and provisioned capacity modes is evaluated per-project based on traffic predictability; DynamoDB Auto Scaling is enabled on provisioned tables. DynamoDB Streams feed Lambda functions for event-driven processing pipelines. It is explicitly not used as a general-purpose replacement for PostgreSQL on projects where relational access patterns are required.

5.6 Snowflake

Snowflake is used as the data warehousing and analytical query layer for clients with established data platform requirements. Snowflake's architectural separation of compute and storage allows query concurrency to be scaled independently of data volume, a property that is commercially important for analytics platforms serving variable numbers of concurrent business intelligence users. Time Travel enables recovery of accidentally deleted or overwritten data up to ninety days in the past, providing a safety net for data engineering pipelines where transformation errors occasionally corrupt output tables. Snowflake Data Sharing is used on multi-tenant analytics platforms where one client's processed data needs to be surfaced to another party without copying it. dbt (data build tool) is the standard transformation layer on all Snowflake projects.

Primary RDBMS	PostgreSQL (85% of projects)
Document Store	MongoDB (schema-justified use cases only)
Cache / Session	Redis (near-universal secondary component)
Search / Analytics	Elasticsearch + Kibana
Serverless / AWS-native	DynamoDB (single-table design pattern)
Data Warehouse	Snowflake + dbt

6. Cloud & Infrastructure

6.1 AWS (Primary Cloud Provider)

Amazon Web Services is ABC Company's primary cloud provider, and the team holds twelve AWS Certified Solutions Architect certifications across Associate and Professional levels. AWS is the default deployment target for all greenfield projects where the client has no prior cloud commitment. The core services used on the majority of engagements include EC2 and Auto Scaling Groups for stateful workloads, ECS with Fargate for containerised services where Kubernetes overhead is not warranted, Lambda for event-driven compute and scheduled tasks, RDS for managed PostgreSQL and MySQL, S3 for object storage and static asset hosting, CloudFront as the CDN and WAF layer, SQS for durable message queuing, SNS for fanout event distribution, Route 53 for DNS management, IAM for access control with least-privilege policies enforced by a custom IAM policy linter in CI, and VPC with private subnets, NAT Gateways, and security groups forming the network foundation. All ABC Company AWS deployments are designed against the AWS Well-Architected Framework's five pillars: operational excellence, security, reliability, performance efficiency, and cost optimisation. Well-Architected Review is offered as a named service to clients.

6.2 Microsoft Azure

Microsoft Azure is used for clients whose identity and directory infrastructure is managed through Azure Active Directory (Entra ID), whose existing investments in Microsoft 365 or Dynamics 365 create integration requirements, or whose compliance posture is certified against Microsoft's compliance portfolio. The services most commonly deployed include Azure App Service for managed web application hosting, Azure Functions for serverless compute, Cosmos DB for globally distributed document storage in Microsoft-native architectures, Azure Service Bus for enterprise messaging, and Azure DevOps for build and release pipelines on clients with existing ADO commitments. Azure Active Directory B2C handles consumer identity on client-facing applications where SSO with Microsoft accounts is required.

6.3 Google Cloud Platform

Google Cloud Platform is used for projects with significant machine learning and data engineering components, where GCP's native ML tooling provides material advantages. BigQuery is the preferred analytical database for data warehouse workloads where Snowflake is not already in use, offering serverless SQL at petabyte scale with a cost model tied directly to bytes scanned. Vertex AI provides the managed model training, evaluation, and serving platform for custom ML models. Cloud Run is used for stateless containerised services requiring scale-to-zero capability with fast cold start, and Google Kubernetes Engine (GKE) is used for orchestrated microservices when the client's broader infrastructure is GCP-based.

6.4 DevOps Tooling

Docker is a universal baseline: every service ABC Company builds runs in a Docker container, and every engineer's local development environment uses Docker Compose to mirror the production service topology. Kubernetes is deployed for microservices architectures at scale, typically managing eight or more services, where deployment coordination, health-based traffic routing, horizontal pod autoscaling, and rolling update strategies justify the operational investment in cluster management. Terraform is the infrastructure-as-code tool for all cloud resources, enabling version-controlled, auditable, reproducible infrastructure across all cloud providers. Terraform Cloud manages remote state and provides a plan-and-apply approval workflow for production changes. GitHub Actions is the primary CI/CD platform, running lint, type-check, unit test, integration test, build, and deploy stages on every pull

request and merge to the main branch. GitLab CI is used for clients who host their code on GitLab. Jenkins pipelines are maintained for legacy client integrations that predate ABC Company's GitHub Actions standardisation, but no new Jenkins pipelines are created.

Tool / Platform	Category	Status	Notes
AWS	Cloud Provider	Primary	12 certified Solutions Architects
Azure	Cloud Provider	Secondary	Microsoft-ecosystem clients
GCP	Cloud Provider	Tertiary	ML/data workloads
Docker	Containerisation	Universal (all projects)	Required baseline
Kubernetes	Container Orchestration	Adopted (8+ service architectures)	EKS / GKE
Terraform	Infrastructure as Code	Universal (all cloud resources)	Terraform Cloud for state
GitHub Actions	CI/CD	Primary	All new projects
GitLab CI	CI/CD	Secondary	GitLab-hosted clients
Jenkins	CI/CD	Legacy (maintenance only)	No new pipelines

7. AI/ML Stack

7.1 LLM Integration

ABC Company's AI engineering practice centres on large language model integration as the primary value-delivery layer for client AI features. OpenAI GPT-4o is the primary model for production deployments: its balance of reasoning quality, context window capacity, multimodal capability, and API reliability meets the requirements of the majority of use cases. Prompt engineering is treated as a first-class software engineering discipline — prompts are versioned in source control, tested against regression suites of known-good expected outputs, and reviewed in code review alongside application code. For tasks requiring extended chain-of-thought reasoning, complex multi-step analysis, or document-length structured output generation, Anthropic Claude Sonnet and Opus models are evaluated as alternatives or complements to GPT-4o. Fine-tuning is used selectively — only when prompt-engineering and retrieval-augmented approaches fail to reach the required quality bar and when sufficient high-quality training data exists. Guardrails are implemented at the application layer using structured output parsing (JSON mode / response format schemas), output classifiers, and content moderation API calls before LLM responses are surfaced to users. Cost optimisation strategies include prompt caching, model routing (using cheaper models for classification and routing tasks before invoking premium models for generation), and aggressive response length controls.

7.2 Orchestration Frameworks

LangChain is used as the LLM orchestration framework for complex, multi-step agentic workflows, tool-calling pipelines, and Retrieval-Augmented Generation architectures. It provides a consistent abstraction over different LLM providers and vector stores, reducing the cost of model substitution. LlamaIndex (formerly GPT Index) is used specifically for RAG pipeline construction, particularly for document ingestion, chunking strategy experimentation, and retrieval evaluation. Hugging Face's Transformers library and the Model Hub are used for accessing open-source models — both for inference via the Inference API on smaller models and for tokenisation utilities and model evaluation in offline pipelines.

7.3 ML Libraries and MLOps

PyTorch is the primary framework for custom model development, experimentation, and fine-tuning, favoured for its dynamic computation graph and Pythonic API that makes debugging significantly more approachable than TensorFlow's graph-based model. TensorFlow and Keras are maintained for existing client models trained prior to ABC Company's PyTorch standardisation. scikit-learn handles classical machine learning requirements — tabular data classification, regression, anomaly detection, and feature engineering pipelines — where deep learning would be architecturally inappropriate. The MLOps layer is built on MLflow for experiment tracking, model registry, and deployment packaging; training pipelines are containerised and orchestrated via Airflow DAGs on AWS MWAA or self-hosted Airflow on EKS.

7.4 Vector Databases and RAG

Qdrant is ABC Company's primary vector database, selected for its performance profile, rich filtering capabilities that allow metadata-filtered approximate nearest-neighbour search, and straightforward self-hosted deployment model on Kubernetes. Qdrant's payload filtering enables hybrid search patterns — combining semantic similarity with structured metadata constraints — that are essential for multi-tenant RAG systems where results must be scoped to a specific organisation, document class, or date range. Pinecone is used when clients require a fully managed vector database service with minimal operational overhead and the simplicity of the managed model justifies the higher per-vector

cost. Embedding strategies are project-specific: OpenAI text-embedding-3-large for English-language document retrieval, multilingual-e5-large for multilingual corpora, and domain-specific fine-tuned embedding models for specialised retrieval tasks where generic embeddings underperform.

7.5 Data Engineering

Apache Airflow is used for data pipeline orchestration — DAG-based scheduling, dependency management, and monitoring of ETL and ML training workflows. AWS Managed Workflows for Apache Airflow (MWAA) is the managed deployment target on AWS, eliminating scheduler and worker management overhead. dbt (data build tool) handles SQL-based data transformation within the data warehouse layer, providing version-controlled, testable, documented transformations with full lineage tracking. Apache Kafka is used for real-time event streaming in architectures where data must flow continuously between services with guaranteed delivery and consumer-group replay capability — Confluent Cloud is the managed deployment target, with self-hosted Kafka on Kubernetes for clients with data-sovereignty constraints.

Component	Primary Tool	Alternative	Use Case
LLM (General)	OpenAI GPT-4o	Anthropic Claude	Generation, reasoning, structured output
LLM Orchestration	LangChain	Custom pipelines	Agentic workflows, tool-calling
RAG Framework	LlamaIndex	LangChain retrievers	Document ingestion, retrieval pipelines
Model Training	PyTorch	TensorFlow / Keras	Custom model development, fine-tuning
Classical ML	scikit-learn	—	Tabular data, classification, regression
Vector Database	Qdrant	Pinecone	Semantic search, RAG retrieval
Experiment Tracking	MLflow	—	Model registry, run tracking
Pipeline Orchestration	Apache Airflow	—	ETL, ML training DAGs
Data Transformation	dbt	—	Warehouse transformation layer
Event Streaming	Apache Kafka	AWS Kinesis	Real-time data pipelines

8. Testing & Quality

8.1 End-to-End Testing

Playwright is ABC Company's primary end-to-end testing framework across all web projects. Its cross-browser support (Chromium, Firefox, WebKit), auto-waiting element interaction model, and first-class TypeScript support make it the most productive E2E tool the team has evaluated. Playwright Test's parallelisation and sharding capabilities allow large test suites to be distributed across CI workers, keeping pipeline durations manageable even as test coverage grows. Cypress is maintained on legacy projects that predate the Playwright migration; no new Cypress suites are started. ABC Company adheres to the test pyramid principle: the majority of test coverage is achieved through unit and integration tests, with E2E tests covering critical user journeys — registration, checkout, authentication, core CRUD flows — rather than attempting to replicate unit-level coverage at the browser level. All E2E test runs are integrated into CI and gate production deployments.

8.2 Unit and Integration Testing

Jest is the unit and integration test runner for all JavaScript and TypeScript projects, paired with React Testing Library for component-level testing. The guiding philosophy is testing observable behaviour from the user's perspective rather than testing implementation details — components are tested by what they render and how they respond to user events, not by inspecting internal state. pytest is the test framework for all Python projects, with pytest-asyncio for async FastAPI route and service testing, and pytest-cov for coverage measurement. All repositories enforce a minimum 80% line coverage threshold in CI, with coverage reports published to the pull request as a comment. Coverage below threshold blocks merge. Integration tests use test containers (Testcontainers library) to spin up real PostgreSQL and Redis instances in Docker for the duration of the test run, ensuring that SQL queries, ORM behaviour, and cache logic are tested against real infrastructure rather than mocks.

8.3 Performance Testing

k6 is used for load testing and performance validation across all API services before production launch and after significant architectural changes. Load profiles are defined per-project based on expected peak traffic: ramp-up, sustained load, spike, and soak scenarios are run against a production-equivalent staging environment. SLA validation scripts define pass/fail thresholds for p95 and p99 response time, error rate, and throughput — a load test that fails its SLA thresholds blocks the release. k6 results are exported to Grafana Cloud for trend analysis across releases.

8.4 Security Testing

Security testing is integrated into the delivery pipeline rather than treated as a pre-release checkpoint. OWASP ZAP is used for Dynamic Application Security Testing (DAST) against staging environments, scanning for injection vulnerabilities, authentication weaknesses, and exposed sensitive endpoints. Snyk is integrated into the GitHub Actions pipeline for Software Composition Analysis (SCA) — scanning all direct and transitive dependencies for known CVEs and blocking merges when high or critical severity vulnerabilities are introduced. npm audit and pip-audit run on every pull request as a lightweight first-pass dependency scan before Snyk's deeper analysis.

Quality Gates (enforced in CI)

- Minimum 80% line coverage (Jest / pytest-cov) — merge blocked below threshold
- Zero high/critical CVEs (Snyk SCA) — merge blocked on introduction
- All Playwright E2E tests passing against staging — blocks production deploy
- k6 load test SLA thresholds met — blocks production deploy on major releases
- OWASP ZAP DAST scan — no new high/critical findings before release

9. Monitoring & Observability

9.1 Application Performance Monitoring

Datadog is ABC Company's primary Application Performance Monitoring platform. Its unified agent model — collecting metrics, traces, and logs through a single deployment — reduces the operational complexity of maintaining separate toolchains for each observability signal. Distributed tracing via Datadog APM automatically instruments NestJS, FastAPI, and Go services, propagating trace context across service boundaries to produce end-to-end waterfall views of request latency. Custom metrics are emitted from application code using the DogStatsD protocol for business-level instrumentation — payment success rates, job queue depths, LLM token consumption — that infrastructure-level metrics cannot capture. Datadog Monitors and Composite Monitors define alerting rules with severity-tiered notification routing. New Relic is used on projects where clients hold existing New Relic contracts.

9.2 Infrastructure Monitoring

For self-hosted and hybrid infrastructure where a SaaS APM platform is not appropriate, ABC Company deploys a Prometheus and Grafana stack. Prometheus scrapes metrics from application /metrics endpoints (using the prometheus-client library for Python and prom-client for Node.js) and from infrastructure exporters — `node_exporter` for host metrics, `postgres_exporter` for PostgreSQL performance counters, and `redis_exporter` for Redis statistics. Grafana dashboards are version-controlled as JSON and deployed via Grafana provisioning, ensuring dashboard definitions are reproducible and code-reviewed. AWS CloudWatch is used for Lambda, ECS, RDS, and other managed AWS services where the Prometheus scrape model does not apply.

9.3 Error Tracking

Sentry is deployed on all web, mobile, and backend services for application error tracking. Source maps are uploaded to Sentry as part of the build pipeline, enabling minified JavaScript stack traces to be de-obfuscated into readable source-level traces in the Sentry UI. Breadcrumbs — the sequence of events, network requests, and console logs preceding an error — are captured automatically, providing the diagnostic context needed to reproduce issues without manual reproduction steps. Sentry Alerts are configured with error rate thresholds and new-issue notifications routed to the relevant engineering team's Slack channel. Performance monitoring in Sentry supplements Datadog for frontend Web Vitals tracking.

9.4 Incident Management

PagerDuty manages on-call rotation schedules and incident escalation for all production services with SLA commitments. Alert sources — Datadog monitors, Sentry issue alerts, and synthetic monitoring failures — route to PagerDuty via integrations, which de-duplicates alerts, applies suppression rules during maintenance windows, and pages the on-call engineer via mobile push, SMS, and phone call in escalating cadence. Post-incident reviews are conducted for all P1 and P2 incidents, with action items tracked to completion and learnings shared across the engineering organisation in a monthly incident digest.

9.5 Logging

Structured JSON logging is enforced across all ABC Company services — log entries are machine-parseable from the point of emission, with consistent fields for trace ID, service name, environment, log level, and request context. For self-hosted logging infrastructure, the ELK Stack (Elasticsearch, Logstash, Kibana) provides ingestion, parsing,

indexing, and visualisation. Logstash pipelines apply field normalisation, PII redaction (email addresses, credit card numbers, authentication tokens), and routing to different Elasticsearch indices based on log classification. For AWS-native deployments, Datadog Logs is used, with Lambda forwarders shipping CloudWatch Logs to Datadog for unified log and trace correlation.

Signal Type	Primary Tool	Self-Hosted Alternative	Scope
Distributed Traces	Datadog APM	Jaeger + OpenTelemetry	All services
Infrastructure Metrics	Datadog Infrastructure	Prometheus + Grafana	All environments
Application Errors	Sentry	Sentry (self-hosted)	Web, mobile, backend
Log Aggregation	Datadog Logs	ELK Stack	All services
Uptime / Synthetics	Datadog Synthetics	Prometheus Blackbox Exporter	All public endpoints
On-Call / Escalation	PagerDuty	—	SLA-committed services
Dashboards (cloud)	Datadog Dashboards	Grafana	Engineering + client-facing
AWS Native Metrics	CloudWatch	—	Lambda, RDS, ECS, managed services